PES UNIVERSITY

Department of Computer Science and Engineering

UE23CS352B — Software Engineering Lab (Lab 5)

Static Code Analysis — Pylint, Bandit, and Flake8

Student Name: Krrish P Raju	**SRN:** PES1UG24CS708
Assigned Script: employee_payroll_system.py (#44)	**Target Repository:** krrish676767/database-query-performance-profiler
Tools Utilized: Bandit 1.9+, Pylint 3.0+, Flake8 7.0+	**Final Pylint Score: 10.00 / 10.00** (Baseline: 5.53/10)

1. Executive Summary & Verification Metrics

Static code analysis involves examining source code without execution to identify security vulnerabilities, logical bugs, and style non-compliance. In this lab, static analysis was performed on the assigned employee_payroll_system.py script. All security vulnerabilities detected by **Bandit** were remediated, code quality measured by **Pylint** was elevated from **5.53/10 to 10.00/10**, and all PEP 8 violations detected by **Flake8** were resolved to zero.

Metric / Analysis Tool	Baseline Scan (Before)	Post-Fix Scan (After)	Lab Requirement Status
Bandit Vulnerabilities	5 (2 High, 1 Med, 2 Low)	**0 Issues (Clean)**	All vulnerabilities resolved
Pylint Code Quality Score	5.53 / 10.00	**10.00 / 10.00 (Perfect)**	Exceeds target (> 9.50/10)
Flake8 PEP 8 Violations	6 violations	**0 Violations (Clean)**	100% PEP 8 compliant
Script Execution	Functional with runtime defects	**Deterministic & Validated**	Expected output verified

2. Issues Identification & Remediation Register

The table below details the prioritized findings across Bandit, Pylint, and Flake8 along with root causes and technical fix approaches:

Issue & Tool Code	Severity	Line	Description & Root Cause	Remediation / Fix Applied
Insecure Eval (B307)	**High** CWE-78	42	eval(deduction_expr) permits arbitrary Python code execution from unvalidated input strings.	Replaced with safe AST recursive parser (_eval_safe_node) whitelisting arithmetic operators only.
Weak Hash MD5 (B324)	**High** CWE-327	47	MD5 is cryptographically broken and vulnerable to collision generation attacks.	Upgraded to secure SHA-256 (hashlib.sha256) with explicit UTF-8 byte encoding.
Shell Injection (B605)	**High** CWE-78	58	os.system('echo ... ' + filename) is vulnerable to command injection via shell metacharacters.	Eliminated subshell invocation; replaced with native Python print() and file logging.
Insecure PRNG (B311)	Low CWE-330	51	random.randint() uses deterministic Mersenne Twister unsuitable for security/financial tokens.	Switched to cryptographically secure CSPRNG (secrets.SystemRandom().randint).
Bare Except (B110/W0702)	Medium CWE-703	26	except: pass silently suppresses all errors including KeyboardInterrupt and logic bugs.	Replaced with explicit defensive dictionary membership and parameter range validation.
Mutable Default Arg (W0102)	Logic Bug	15	transaction_logs=[] creates a persistent list instance shared across all function calls.	Changed default argument to None and initialized dynamically inside the function body.
Resource Leak (R1732/W1514)	Resource Defect	55	Unmanaged open() risks descriptor leaks on exception; missing explicit text encoding.	Enforced context manager with explicit UTF-8: with open(fn, 'w', encoding='utf-8'):.
Dead Code & Unused Variables	Code Quality	6, 31, 41	Unused import math (W0611, F401); dead variables statutory_buffer and base.	Removed unused imports and pruned dead variable assignments completely from execution paths.
PEP 8 Style & Line Length	Style / PEP 8	15, 19, 32	Line length > 79 chars (E501); camelCase naming (C0103); singleton comparison == None (E711).	Wrapped lines < 79 chars; renamed to process_salary with alias; used is None.

3. Analytical Reflection & Engineering Inquiry

Q1: Defect Prioritization & Severity-Driven Triage

Analysis: In enterprise environments, remediation must be guided by risk exposure. Vulnerabilities detected by **Bandit** were triaged first because exploitable security flaws (CWE-78 arbitrary evaluation, command injection, and broken hashing) present catastrophic business liabilities, compliance violations, and immediate attack vectors. Eliminating the eval() vulnerability averted potential Remote Code Execution (RCE). Next, logic flaws flagged by **Pylint**—such as mutable default arguments (transaction_logs=[]) and bare except: pass clauses—were remediated because they directly corrupt operational state and hide fatal defects. Finally, **Flake8** PEP 8 formatting rules and documentation gaps were resolved to ensure long-term readability and maintainability.

Q2: Complementary Synergy of Bandit, Pylint, and Flake8

Analysis: Each static analysis tool operates on a specialized dimension of software quality. **Bandit** parses the Abstract Syntax Tree (AST) specifically for security hazards and Common Weakness Enumeration (CWE) anti-patterns, yet remains agnostic to unhandled logic flaws or style. **Pylint** performs deep semantic inspection, computing structural cyclomatic complexity, uncovering unclosed OS handles, dead code, and mutable defaults. **Flake8** provides blistering-fast grammar and style enforcement against PEP 8, catching line overflows and improper spacing without deep semantic overhead. Combining all three creates an indispensable defense-in-depth static quality pipeline.

Q3: Technical Challenges, Trade-offs & Architectural Remediation

Analysis: The most technically demanding refactoring involved safely computing deduction formulas without eval(). Simply disabling eval() would break business logic requiring dynamic percentage calculations (e.g., 'base * 0.85'). To resolve this, an AST-based recursive node evaluator (_eval_safe_node) was implemented to whitelist only approved arithmetic binary operators (+, -, *, /) and numeric literals, rejecting arbitrary function execution. Additionally, to resolve Pylint's snake_case naming enforcement while avoiding breaking downstream consumers, the function was renamed to process_salary and a backward-compatible alias processSalary = process_salary was exported.

Q4: Integration into CI/CD Pipelines and Modern DevOps Workflows

Analysis: Incorporating static code analysis into Continuous Integration (CI/CD) pipelines automates quality governance at scale. By establishing pre-commit hooks and Pull Request gatekeepers (e.g., blocking merges if Bandit finds High/Medium vulnerabilities or if Pylint falls below 9.5), organizations enforce 'shift-left' security. This minimizes technical debt accumulation, slashes human code review fatigue by automating style verification, and ensures cross-platform determinism across development teams.

4. Automated Verification Results (Post-Remediation)

Confirmation of static analysis tools executed post-fix on employee_payroll_system.py:

Static Tool	Command Executed	Exit Code	Report Summary / Final Score
Bandit	`bandit -r employee_payroll_system.py`	0	**No issues identified** (0 High, 0 Medium, 0 Low)
Pylint	`pylint employee_payroll_system.py`	0	**Rated at 10.00 / 10.00** (Clean execution)
Flake8	`flake8 employee_payroll_system.py`	0	**0 Violations** (Full PEP 8 compliance)